

DnDRom workspace interaction and loading

Findings and design direction

DnDRom needs a task-oriented editor with a stable central viewport, immediately available assets, and direct manipulation that shares the renderer's camera. The existing dark fantasy materials, typography, and gold accents can remain. The structural problem is competition between unrelated tasks inside the same narrow, height-limited panel.

The Build sidebar currently places connected-world planning, journey controls, and weather above the library. A paused generation request expands this content until the asset area loses its useful height. The Create popup also lives inside that panel's stacking and clipping hierarchy. These are layout ownership failures: making the thumbnails smaller or raising one z-index does not establish a reliable workspace.

Nielsen Norman Group's progressive-disclosure guidance supports keeping frequent actions immediately available and placing specialized work in clearly labeled secondary areas. Its tab guidance distinguishes switching between related views from launching unrelated actions. These principles support persistent Assets, World, and Environment sections, plus a separate Create action dialog.^{1, 2}

A second ownership problem affects transformations. A browser overlay periodically requests projected points from Unreal, then draws them on a different frame. Meanwhile, each drag update goes through campaign mutation and persistence. The result can be a responsive cursor with delayed geometry and a handle detached from the rendered object. Visible handles should be rendered by Unreal; browser controls can remain responsible for task selection and pointer capture.

Loading also has distinct stages. Accepted scene plans, evolved terrain fields, compiled mesh payloads, native mesh resources, and GPU pipelines are different caches. Possessing a seed or an accepted plan does not mean the final rendered representation is ready. Each layer needs explicit reuse and independent timing.

Workspace information architecture

The proposed organization keeps navigation stable while giving each destination a different working surface. Build prioritizes finding and placing assets. Play prioritizes party state and current objectives. Scene prioritizes describing and reviewing a location. Prop prioritizes references and placement properties. Character prioritizes artwork, the miniature, and its base. Dice prioritizes surface appearance and a live die preview.

Destination	Primary left area	Center	Contextual right area
Build	Searchable Assets by default; World and Environment sections	Editable world	Selected object's properties
Play	Party, health, active objectives	Playable scene	DM, party, and session controls
Scene	Description and generation action; expandable world/region settings	Concepts and native scene preview	Style references and publish controls
Prop	Description or reference import	Reference and model review	Placement, collision, material properties, save
Character	Artwork, name, sheet, generation/import	Miniature preview	Base and scale first; restyling and rigging disclosed separately
Dice	Surface description and artwork	Die preview	Shape and material; optional effects

These are implementation recommendations derived from the product's tasks, rather than findings that a usability study has already validated. Test them with actual creation and placement tasks. A successful layout must permit a user to find an existing asset without dismissing a failed world-generation request, and to return to that request without losing the asset search.

Panels should have one clear scroll owner. Headers, section controls, and primary navigation must not shrink. The content region takes the remaining height with an explicit zero minimum, and its grid scrolls internally. On shorter displays, disclosure controls reduce the amount of advanced content competing with the preview. On narrow displays, existing collapsible side panels remain necessary; reducing the viewport below a usable width is not an acceptable substitute.

The Create chooser is an action dialog rather than a faux navigation tab. Each action explains its destination. It is mounted outside the clipped sidebar, receives focus, traps keyboard focus, closes with Escape, and returns focus to its trigger. Those behaviors follow WAI's modal-dialog pattern.³

Navigation and accessibility

Tabs must identify the active section visually and programmatically. Each tab names its associated panel. Left and Right arrows move between tabs; Home and End reach the ends. Tab leaves the tab list for the active panel. WAI's pattern supplies these mechanics and avoids a screen reader encountering several apparently active sections.⁴

Search and category filtering remain within Assets, so switching to World does not clear them. Catalogue remains the explicit place to browse and manage saved libraries. Create remains an explicit action. These labels should not be repurposed for different meanings in different areas.

Errors belong next to the affected operation and may expand within that section's scroll region. A failed harbor plan should remain reviewable in World, but should not displace Build's asset grid. Generation activity can continue globally while the user changes tasks. The UI must distinguish a paused planner from a failed renderer and a still-loading scene.

Character and Dice have long sequences of optional settings. Disclosure is preferable to hiding required steps. Artwork is immediately available, with the optional character sheet in a disclosure. Base shape, footprint, model scale, and save remain visible together in Character; style edits and rigging are secondary tasks. Dice keeps material generation beside the description, with external templates inside a secondary disclosure. Scene keeps its description and Generate action prominent, while region seed and advanced generation choices can be opened as needed.

Large buttons alone do not establish accessibility. Keyboard focus, meaningful names, visible state, sufficient contrast, and escape behavior all need checks in the embedded browser. Screen captures establish layout evidence; they cannot establish full assistive-technology compatibility.

Direct manipulation and transform handles

The editor should offer distinct move arrows, rotation rings, scale boxes, and planar move squares. Axis colors remain consistent across tools. The central ground handle moves over actual terrain. Pointer capture keeps a drag active when the cursor leaves the narrow handle. Snapping is a modifier, not a requirement for every movement.

Blender's published transform-gizmo documentation establishes the move/rotate/scale convention, although the accessible indexed version is old and is used here only for that stable convention.⁵ Ryan Schmidt's runtime tools discussion describes Unreal's Interactive Tools Framework and the significance of communicating view state between game and render systems. It also warns that engine versions change runtime geometry interfaces; the 2022 article is architectural evidence rather than a drop-in UE 5.8 implementation guide.⁶

The current repair draws visible primitives in Unreal's frame loop using its current camera. Browser-projected shapes remain transparent hit targets and refresh as quickly as the bridge allows, with at most one projection request outstanding. This removes the visible 80 ms projection cadence. It does not imply that browser pointer hit testing has zero latency.

During a drag, the browser sends bounded native transform previews. It does not serialize the full campaign for every pointer event. One latest pending preview replaces obsolete intermediate positions. Native code applies each preview relative to its last applied transform. On release, a durable scene edit commits the final result, using that native preview state to avoid applying the same movement twice. Cancellation restores the initial transform.

The distinction between preview and commit is important for performance and correctness. Scene changes remain persistent, while a temporary pointer movement cannot force terrain compilation, material import, or fog restart. Selection IDs must continue to resolve to the actual owning native actor, including objects added after initial scene import.

Nielsen's response-time guidance identifies roughly 0.1 seconds as an important threshold for perceived immediate response. A modern 3D editor should aim for much tighter, frame-level feedback; the historical threshold is a perceptual guideline rather than permission to add 100 ms of delay.⁷

Loading architecture and comparison with packaged games

Epic's level-streaming documentation describes loading and unloading portions of a world during play. World Partition organizes actors into spatial cells and streams them around sources. It does not require an entire large world to exist at its highest detail at once.^{8, 9}

DnDRom should offer the same perceptual result: useful local surroundings first, coherent coverage beyond them, and additional detail arriving without disrupting play. However, its generated geometry adds an expensive first-time computation stage. The relevant comparison with a packaged game is the revisit path: once the local world has been generated, the application should read its saved renderable representation instead of evolving the terrain again.

The inspected loader has a serial procedural worker. A cache miss starts synchronous erosion before later messages can restore cached tiles. Thus a valid cache can exist while the display still appears to regenerate. The repair separates cache probing from expensive generation: restore completed payloads first, then process misses. The queue remains bounded so a large atlas does not create an unbounded number of pending native imports.

The erosion cache has a related spatial problem. Startup reads only fields near the initial focus, but distant tile sampling is synchronous. Fields already on disk outside that initial set cannot be reached in time and are recomputed. Before building a missing tile, prefetch its regional and detail dependencies, including the regional apron used by detail fields. This preserves the erosion equations and world geography while changing when saved data becomes available.

The full-town validation exposed a further admission failure: the combined local payload exceeded the per-entry limit and therefore was never cached. Persist each bounded scene batch separately, then atomically publish a small index naming all completed parts. On read, require all referenced batches; a partial index cannot represent an accepted render. Existing single-entry records remain readable.

Large native imports also receive a content-addressed disk cache. The current stream identity is rebound when a validated payload is read in a new session, so a random upload ID does not force another large browser transfer. Native scene parsing and resource creation still occur; this is not a serialized GPU-resource cache.

Compiled local-scene payloads need a separate disk allowance from distant terrain. Otherwise atlas writes can evict the playable town immediately before the next restart. Compress large geometry strings on disk while preserving typed material buffers. Reads accept existing uncompressed records; a new representation must not invalidate usable old caches merely to change storage format.

Epic's asynchronous asset-loading guidance also warns that hard references can cause large asset sets to load at startup. Soft references and asynchronous requests are useful for cooked reusable assets, but adding those APIs alone does not eliminate custom mesh generation or browser/native serialization.¹⁰

Memory, cache, and scheduling policy

The repair bounds compiled local geometry to a 384 MiB disk pool and distant compiled payloads to a separate 768 MiB pool. Geometry strings are compressed. Individual source payloads above 256 MiB are not inserted into this cache. These are disk budgets, not guaranteed RAM consumption or GPU allocations. A separate native payload cache retains at most one GiB of validated JSON imports, evicting older files by access time.

Erosion records remain typed numerical fields. The persistent allowance increases from 192 to 768 records, while the worker prefetch map is bounded to 192. At the present 257-by-257 field resolution and four arrays, 768 records are roughly one GiB before database overhead. These fields are regenerable and separate from authoritative campaigns. Multiple worker-local samplers also retain fields, so process memory must be measured rather than inferred from this one limit.

Accepted world plans and authored object changes remain authoritative. Cache eviction is allowed to increase load time; it must not change terrain samples, parcel IDs, or object layouts. Cache keys must include geography, origin, generator context, and meaningful revisions. Ordinary token transforms must not invalidate regional terrain.

Metrics should separate campaign hydration, asset reads, local compilation or cache decode, native upload, mesh/resource creation, collision readiness, first playable frame, surrounding coverage, and background completion. A 15 ms cached worker result is not a 15 ms application startup. It excludes process launch, image reads, native resource creation, and graphics-driver work.

Pipeline-state compilation is another possible source of hitches. Epic's PSO precaching documentation describes asynchronous pipeline preparation and validation. It should be investigated through profiling if hitches remain after the demonstrated scene-rebuild and queue problems are removed. Enabling an unrelated setting without measuring the bottleneck would not establish a loading improvement.¹¹

Loading mist and the horizon

Loading reveal is temporary presentation state. Weather fog and aerial perspective are persistent environmental effects. They need different ownership and completion rules.

The inspected reveal waits on a large atlas selection extending roughly 160 km. The shader can also integrate mist over low-angle sky rays, creating a horizontal curtain. Neither is required to hide nearby terrain creation. A fixed reveal cohort around the scene origin makes the endpoint stable during camera orbit. Acknowledged coarse coverage counts as landscape; later refinement must not restart the reveal.

The repair uses a 12 km nearby completion region and smoothly removes the loading effect across its outer 10?12 km band. Sky-depth receivers are excluded. This is a product and rendering policy, not a physical atmospheric model. The accepted water and weather systems remain separate. Beyond that range, ordinary atmospheric perspective provides distance context while the atlas continues its work.

The completion test still rejects holes inside the reveal region. It accepts a ready parent covering all its children, because that parent is rendered scenery. An incomplete far region outside the cohort cannot hold loading fog indefinitely. This tradeoff needs ground-level and elevated-camera validation: background refinement can remain observable beyond the mist's range, and fog must not be used to disguise missing playable geometry.

A correct reveal test should assert both state and pixels: opacity reaches zero, camera orbit does not restore it, the sky remains visible during loading, and no black band survives at the horizon. A timer that merely disables fog after a fixed delay would not prove coverage or solve queue starvation.

Acceptance matrix and limitations

Check	Evidence required
Assets visible with a failed planner	Short-window capture, clickable cards, scrollable library
Create chooser usable	All actions visible, pointer activation, Escape and focus return
Context-specific workshops	Each top-level tab inspected; preview and primary action usable
Live movement	Native transform changes before mouse release; mid-drag image
Planar and rotational movement	Two-axis motion and constrained rotation tested separately
Camera/handle registration	Orbit captures from the native renderer; no delayed visible browser copy
Revisit reuse	Fresh process, same saved fixture, cache hits and zero repeated erosion where data exists
Cache correctness	Compressed and legacy payloads restore materials and geometry
Reveal termination	Loaded nearby cohort reaches zero opacity and stays clear on orbit
Delivery	Extracted installer matches tested files; project icon and previous installer preserved

The minimum configuration remains 8 GB VRAM and 16 GB RAM, with 1080p/30 FPS as a target. This document does not claim that target has been established. Test representative hardware with the local AI runtime active and report frame-time percentiles and peak memory separately from geometry-worker timings.

The repair does not implement a complete Blender clone, a new world-partition engine, or a replacement water shader. It targets the demonstrated sidebar ownership, delayed manipulation, cache starvation, and reveal-lifetime faults. Broader native editing features can build on the same preview/commit contract without making durable world changes part of the pointer loop.

Sources

1. Jakob Nielsen, Nielsen Norman Group. [Progressive Disclosure](#). December 3, 2006. Primary usability guidance; stable interaction principles.
2. Nielsen Norman Group. [Tabs, Used Right](#). Current article accessed September 14, 2026. Tab semantics and navigation structure.
3. W3C WAI. [Dialog \(Modal\) Pattern](#). Living guidance accessed September 14, 2026. Focus containment and dismissal.
4. W3C WAI. [Tabs Pattern](#). Living guidance accessed September 14, 2026. Keyboard and semantic requirements.
5. Blender Foundation. [Transformation Gizmos, Blender 2.90 Manual](#). Historical indexed manual; full page fetch unavailable. Used only for the move/rotate/scale convention, not current API claims.
6. Ryan Schmidt. [The Interactive Tools Framework in UE5](#). June 22, 2022. First-party framework implementation discussion; version limitations acknowledged.
7. Jakob Nielsen, Nielsen Norman Group. [Response Times: The 3 Important Limits](#). January 1, 1993. Perceptual response-time guidance.
8. Epic Games. [Level Streaming Overview](#). UE 5.8 documentation accessed September 14, 2026.
9. Epic Games. [World Partition](#). UE 5.8 documentation accessed September 14, 2026.
10. Epic Games. [Asynchronous Asset Loading](#). UE 5.8 documentation accessed September 14, 2026.
11. Epic Games. [PSO Precaching](#). UE 5.8 documentation accessed September 14, 2026.